ShortsAI 코드 안내서

초보자를 위한 코드 구조, 언어, 도구, 연결 방식 설명 · v0.9.2 기준 · 2026-10-13

> **이 문서의 목표:** 코딩을 깊게 몰라도 "어떤 파일이 무슨 일을 하고, 서로 어떻게 연결되는지"를 이해하고, 간단한 수정은 어디를 고치면 되는지 찾을 수 있게 하는 것이다.
>
> **읽는 순서 추천:** 1부(용어, 큰 그림) → 2부(폴더 지도) → 3부(쇼츠 하나 열었을 때 일어나는 일) → 필요한 파일만 4~6부에서 찾아보기 → 9부(자주 하는 수정)

목차

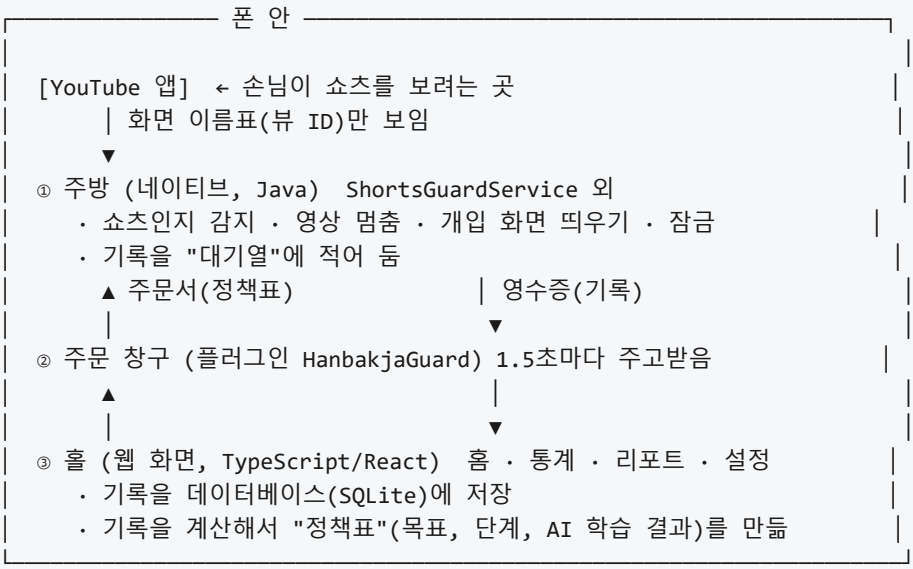
1. 1부. 먼저 알아 둘 용어와 큰 그림
2. 2부. 사용한 언어와 도구, 폴더 지도
3. 3부. 쇼츠를 하나 열었을 때 코드가 움직이는 순서
4. 4부. 휴대폰 쪽 코드 (Java) 파일별 설명
5. 5부. 앱 화면 쪽 코드 (TypeScript) 파일별 설명
6. 6부. 둘을 잇는 방법 (플러그인, 동기화, 정책표)
7. 7부. 데이터는 어디에 어떻게 저장되나
8. 8부. 설정 파일, 빌드, 테스트
9. 9부. 자주 하는 수정은 어디를 고치나

1부. 먼저 알아 둘 용어와 큰 그림

1. 꼭 필요한 용어

용어	쉬운 설명
코드 / 소스 코드	사람이 읽을 수 있게 적은 프로그램 설계도. 이 프로젝트의 `.java`, `.ts`, `.tsx` 파일들
빌드	설계도(코드)를 실제로 돌아가는 프로그램으로 바꾸는 작업. 결과물이 APK
APK	안드로이드 앱 설치 파일. 폰에 깔리는 "완성품"
네이티브 (Native)	안드로이드가 원래 쓰는 방식으로 만든 코드. 폰의 깊은 기능(다른 앱 화면 감지, 소리 제어)을 쓸 수 있다. 이 프로젝트에서는 Java
웹 (Web)	웹사이트를 만드는 기술(HTML, CSS, JavaScript). 화면 만들기가 빠르지만 폰의 깊은 기능은 못 쓴다. 이 프로젝트의 앱 화면(홈, 통계, 설정)
하이브리드 앱	웹으로 만든 화면을 앱 안에 넣고, 필요한 부분만 네이티브로 만든 앱. ShortsAI가 이 방식
접근성 서비스	원래 시각장애인 등을 돕기 위해 다른 앱의 화면 구조를 읽을 수 있게 해 주는 안드로이드 기능. 우리는 이걸로 "지금 쇼츠 화면인지"를 알아낸다
뷰 ID (View ID)	앱 화면 안의 각 부품에 붙은 이름표. 예: YouTube 쇼츠 플레이어의 이름표는 `reel_recycler`. 우리는 글자 내용은 안 보고 이 이름표만 본다
오버레이	다른 앱 화면 위에 겹쳐 띄우는 창. 쇼츠 위에 뜨는 개입 화면
함수 (function, 메서드)	"이 일을 해라"를 묶어 이름 붙인 코드 덩어리. 예: `decide()` = 개입을 어떻게 할지 정하는 함수
컴포넌트 (component)	화면의 한 조각(버튼, 카드, 패널)을 만드는 코드. React에서 쓰는 단위
플러그인	웹 코드가 네이티브 기능을 부를 수 있게 해 주는 연결 장치
JSON	데이터를 글자로 적는 약속된 형식. 예: `{"goal": 20, "aiEnabled": true}`. 웹과 네이티브가 이 형식으로 주고받는다
SQLite	폰 안에 만드는 작은 데이터베이스(표 모음). 사용 기록을 저장
SharedPreferences	안드로이드가 주는 작은 메모장. 네이티브 쪽이 잠깐 들고 있는 값(대기 중인 기록, 잠금 시각 등)을 적어 둔다
테스트	코드가 맞게 동작하는지 자동으로 확인하는 코드. 고칠 때마다 돌려서 실수를 잡는다
git, 커밋	코드 변경 기록을 남기는 도구. 커밋 = "이 시점을 저장"

2. 큰 그림: 식당으로 비유하면



- **주방(네이티브)**은 손님(사용자) 바로 앞에서 일한다. 쇼츠가 열리는 순간 반응해야 해서 Java로 만들었다.
- **홀(웹)**은 통계, 설정 같은 화면을 보여 주고, 기록을 정리하고, AI 계산을 한다.
- **주문서(정책표)**: 홀이 "목표 20분, 이 시간엔 호흡이 잘 먹힘" 같은 지시를 적어 주방에 넘긴다. 주방은 이 표만 보고 개입한다.
- **영수증(기록)**: 주방이 "몇 시에 무슨 개입을 했고 사용자가 멈췄는지"를 적어 홀로 보낸다. 홀이 데이터베이스에 저장하면 주방은 영수증을 지운다.

2부. 사용한 언어와 도구, 폴더 지도

3. 언어

언어	어디에	양	쉬운 설명
Java	android/app/src/main/java/...	약 4,100줄	안드로이드 기본 언어. 쇼츠 감지, 개입 화면, 잠금
TypeScript	lib/	약 3,000줄 (+테스트 1,070줄)	JavaScript에 "자료형 검사"를 더한 언어. 숫자 자리에 글자를 넣는 실수를 미리 잡아 준다. 계산과 저장 담당
TSX (TypeScript + React)	components/, app/	약 4,000줄	화면을 만드는 TypeScript. HTML처럼 생긴 문법을 섞어 쓴다
XML	android/app/src/main/res, AndroidManifest.xml	적음	안드로이드 설정 파일 형식
CSS (Tailwind)	app/globals.css, 각 화면의 className	적음	모양(색, 크기, 간격). Tailwind는 `className="text-sm p-4"` 처럼 짧은 이름으로 모양을 정하는 도구
SQL	lib/db.ts 안	적음	데이터베이스에 표를 만들고 읽고 쓰는 언어
JavaScript (Node)	scripts/*.mjs	약 270줄	빌드 자동화 스크립트
Python	scripts/make-icons.py	60줄	앱 아이콘 이미지 만들기

4. 도구

도구	하는 일
Next.js 16	React로 만든 화면을 페이지(홈, 통계, 설정 ...)로 나누고, 앱에 넣을 수 있는 파일로 내보낸다
React 19	화면을 컴포넌트(조각)로 나눠 만드는 도구. 데이터가 바뀌면 화면이 자동으로 다시 그려진다
Capacitor 8	웹 화면을 안드로이드 앱으로 감싸고, 웹과 Java를 잇는 플러그인을 제공
Gradle	안드로이드 코드를 APK로 빌드하는 도구 (`android/gradlew.bat`)
npm / Node.js	웹 쪽 라이브러리 설치(`npm install`), 명령 실행(`npm run ...`)
Android Studio	안드로이드 개발 프로그램. SDK(안드로이드 도구 모음)와 JDK 21 제공
Claude Code	AI 코딩 도구. 코드 작성, 수정, 테스트 실행
git, GitHub	버전 기록, 코드와 APK 공유
ADB	PC와 폰을 연결해 로그를 보는 도구 (무선 디버깅)
shadcn/ui, Base UI	버튼, 스위치 같은 화면 부품 모음 (`components/ui/`)
@capacitor-community/sqlite	폰 안 SQLite 데이터베이스를 웹 코드에서 쓰게 해 주는 플러그인
@capacitor/local-notifications	서버 없이 폰이 직접 보내는 알림 (매일 리포트, 위험 시간 알림)

5. 폴더 지도

```
ShortsAI/
├ android/                          ← 안드로이드 프로젝트 (네이티브)
│  ├ app/
│  │  ├ build.gradle               ← 앱 빌드 설정 (버전, 서명)
│  │  └ src/
│  │     ├ main/
│  │     │  ├ AndroidManifest.xml   ← 앱 등록증 (권한, 서비스 등록)
│  │     │  ├ java/app/focuson/shorts/   ← ★ Java 코드 13개 (4부)
│  │     │  ├ res/xml/accessibility_service_config.xml ← 접근성 서비스 설정
│  │     │  └ assets/public/      ← 빌드된 웹 화면이 복사되는 곳 (자동 생성)
│  │     └ test/java/...          ← Java 자동 테스트
│  └ gradlew.bat                  ← 빌드 도구 실행 파일
├ app/                             ← 웹 페이지 주소별 파일 (/, /stats, /report, /settings, /watch)
├ components/                      ← ★ 화면 조각 (5부)
│  └ ui/                          ← 버튼, 스위치 같은 기본 부품
├ lib/                             ← ★ 계산, 저장, AI (5부)
├ scripts/                         ← 빌드 자동화 (APK 만들기 등)
├ docs/                            ← 문서 (인수인계, PDF)
├ release/                         ← 만든 APK와 테스트 목록(NOTES.md)
├ public/                          ← 아이콘 등 그대로 쓰는 파일
├ package.json                     ← 웹 쪽 라이브러리 목록, 명령, ★앱 버전
├ capacitor.config.ts              ← Capacitor 설정 (앱 ID, 웹 폴더 위치)
└ next.config.ts                   ← Next.js 설정 (정적 파일로 내보내기)
```

> **팁:** `node_modules/`, `out/`, `.next/`, `android/app/build/` 폴더는 빌드할 때 자동으로 생기는 것이라 고치지 않는다. git에도 올라가지 않는다.

3부. 쇼츠를 하나 열었을 때 코드가 움직이는 순서

사용자가 YouTube에서 쇼츠를 연 순간부터, 기록이 통계에 보일 때까지를 따라가 본다. 괄호 안은 **파일 이름 → 함수 이름**이다.

① 화면이 바뀌었다는 신호를 받는다

안드로이드는 화면이 바뀔 때마다 접근성 서비스에 "이벤트"를 보낸다. (`ShortsGuardService.java → onAccessibilityEvent()`)

② 쇼츠 화면인지 판단한다

지금 앞에 있는 창의 이름표(뷰 ID)들을 모아서 쇼츠 플레이어인지 본다. `reel_recycler` 같은 쇼츠 이름표가 보이고, 일반 영상 이름표(`watch_player` 등)가 없으면 쇼츠다. (`classifyActiveWindow()` → `GuardScreens.java`)

③ "들어왔다 / 나갔다"로 정리한다

스와이프할 때마다 신호가 와도 "쇼츠에 들어온 것"은 한 번만 센다. 쇼츠 화면이 잠깐 사라지면 1초 기다렸다가 다시 확인한다. (`ShortsSessionTracker.java`)

④ 무엇을 할지 정한다

`ShortsGuardService.java → decide()`가 순서대로 확인한다.

1. 잠금 중이면 → "쇼츠 쉬는 중" 화면 (`GuardFlow.locked()`)
2. 10분 안에 돌아온 약속이 있으면 → 남은 약속 이어가기
3. 개입 스위치가 꺼져 있으면 → 기록만 한다
4. 아니면 → 영상을 멈추고(`pausePlayback()`: 오디오 포커스 요청 + 음소거), 오늘 사용량으로 단계를 정해 개입 화면을 띄운다 (`GuardFlow.entry()` 또는 `GuardFlow.strong()`)

⑤ 개입 화면을 단계별로 보여 준다

`GuardFlow.java`가 "방식 → 이유 → 약속"을 차례로 보여 준다. 방식은 AI가 켜져 있으면 `GuardBandit.pick()`이, 꺼져 있으면 차례대로 고른다. 화면을 실제로 그리는 건 `GuardOverlay.java`, 문구는 `GuardCopy.java`.

⑥ 사용자가 고른다

- **나가기** → 칭찬 → YouTube에서 나감 (`finishExit()` → `leavePlayer()`)
- **N분 보기** → 소리를 되돌리고 타이머 시작 (`scheduleCommitment()`). 시간이 되면 다시 멈추고 묻는다 (`GuardFlow.commitmentOver()`)

⑦ 기록을 대기열에 적는다

무슨 개입을 했고 결과가 어땠는지 JSON으로 적어 `GuardState.addIntervention()`으로 대기열에 넣는다. 시청 시간은 `GuardState.endSession()`.

⑧ 웹이 가져가서 저장한다

앱 화면이 켜져 있으면 1.5초마다 `components/guard-bridge.tsx`가 플러그인으로 대기열을 가져와(`sync`) 웹 형식으로 바꾸고(`lib/logic.ts → convertNativeBatch()`) SQLite에 저장한 뒤(`lib/db.ts → applyOps()`) "저장했으니 지워도 돼"(`ack`)를 보낸다.

⑨ 다음 개입을 위한 정책표를 만들어 넘긴다

같은 sync 때 `lib/policy.ts → buildPolicy()`가 목표, 단계 기준, AI 학습 결과(`lib/ai.ts → banditTables()`)를 JSON 정책표로 만들어 네이티브에 넘긴다. 네이티브는 다음 개입부터 이 표를 쓴다.

⑩ 화면에 보인다

홈, 통계, 리포트 화면(`components/home-view.tsx` 등)이 저장된 기록으로 그래프와 숫자를 그린다.

> **중요:** 앱 화면이 꺼져 있어도 ①~⑦은 네이티브만으로 계속 동작한다. 기록은 대기열에 쌓여 있다가, 다음에 앱을 열면 ⑧에서 한꺼번에 저장된다.

4부. 휴대폰 쪽 코드 (Java) 파일별 설명

위치: `android/app/src/main/java/app/focuson/shorts/`

6. 감지와 흐름의 중심

ShortsGuardService.java (약 940줄) — 앱의 심장

접근성 서비스 본체. 안드로이드가 백그라운드에서 계속 살려 두는 부분이다.

함수	하는 일
onServiceConnected	서비스가 켜질 때 한 번. 준비 작업, 꺼지기 전 남은 기록 정리, 소리 원래대로
onAccessibilityEvent	화면이 바뀔 때마다 호출. 너무 자주 오므로 잠깐 모았다가 `evaluate()`로 판단
tick	1초마다 실행. 화면 꺼짐, 다른 앱 이동, 작은 창(PIP) 확인, 시청 중 시각 갱신
classifyActiveWindow / classifyYoutube	앞 화면의 이름표를 모아 쇼츠인지 판정
decide	쇼츠에 들어왔을 때 무엇을 할지 정함 (3부 ④)
pausePlayback / restoreAudio	영상 멈춤(오디오 포커스 + 음소거), 소리 되돌리기
scheduleCommitment / onCommitmentOver	"N분 보기" 타이머. 쇼츠를 나가면 멈추고 돌아오면 이어감
leavePlayer / stepAway	쇼츠에서 나가기 (뒤로 가기, 안 되면 홈)
youtubePipVisible / checkPipLeave / pipStillOpen	작은 창 감지와 처리 (한 번만 펼치기 시도)
log	디버그 APK에서만 로그(`ShortsAIGuard`) 출력. 다른 앱 이름은 남기지 않음

GuardScreens.java (143줄) — 쇼츠 판별 규칙

"이 이름표들이 보이면 쇼츠, 이 이름표가 있으면 일반 영상" 같은 규칙만 모은 파일. 안드로이드 기능을 안 써서 자동 테스트로 확인하기 쉽다.

```
쇼츠 이름표:     com.google.android.youtube:id/reel_recycler
                 com.google.android.youtube:id/reel_player_page_container
일반 영상 이름표: watch_player, watch_panel, watch_list, player_overlays …
→ 쇼츠 이름표가 보이고 일반 영상 이름표가 없을 때만 쇼츠
```

ShortsSessionTracker.java (103줄) — 머문 시간 정리

화면 신호를 "쇼츠에 머문 한 번"으로 묶는다. 스와이프로 다음 영상을 봐도 같은 한 번으로 세고, 다른 앱이 앞에 오면 바로 끝낸다. 끝난 시각은 "쇼츠가 처음 안 보인 시각"으로 정확히 잡는다.

7. 개입 화면

GuardFlow.java (약 870줄) — 개입 화면의 순서

개입 화면에 어떤 단계를 어떤 순서로 보여 줄지 정한다. 실제 그리기는 GuardOverlay에 맡긴다.

함수	하는 일
entry	약함·중간 단계 진입: 방식 → 이유 → 약속
method	방식 고르기. 공부 모드면 카드 먼저, AI가 켜져 있으면 밴딧, 아니면 차례대로
pause / framing / breath / alternative / reflection / card	방식 6가지 각각의 화면
cardView	공부 카드 보기 버튼, 정답 표시, 해설, "이 문제 이상해요"
reason / commitment	이유 고르기, "몇 분 볼래요?" (위험 시간이면 최대 3분)
commitmentOver	약속 시간 끝: 나가기 또는 조금 더
strong / strongChoices / overdraftNudge	목표 초과: 오늘은 끝, 잠그기, 3분만 더(대기 화면)
locked	잠금 중 화면 (남은 시간, 공부 카드)
praiseThen / feedbackRow	칭찬, 5번에 1번 "도움이 됐나요?"
Rec (안쪽 클래스)	개입 한 번의 기록을 담는 상자. 끝나면 JSON으로 바꿔 대기열에 넣는다

GuardOverlay.java (362줄) — 화면 그리기

쇼츠 위 아래쪽에 반쯤 올라오는 어두운 창(바텀 시트)을 코드로 직접 그린다. 버튼, 글자, 게이지, 칩(작은 선택 버튼)을 만드는 도구 함수가 모여 있다. `TYPE_ACCESSIBILITY_OVERLAY`라는 창 종류를 써서, "다른 앱 위에 그리기" 권한 없이 접근성 권한만으로 띄운다. 글자 입력칸은 일부러 만들지 않았다.

GuardCopy.java (271줄) — 화면 문구 모음

개입 화면에 나오는 모든 문장이 여기 있다. 사용자(팀)가 상황별 표로 확인한 문구만 넣는다. **문구를 바꾸려면 이 파일을 고친다.**

8. 상태, 정책, AI

GuardState.java (약 700줄) — 네이티브의 메모장

SharedPreferences에 값을 적고 읽는 함수 모음. 앱이 꺼졌다 켜져도 남아야 하는 것을 여기에 둔다.

- 대기열: 시청 기록(`endSession`), 개입 기록(`addIntervention`), 잠금 중 시도. 웹이 저장하고 `ack`하면 지운다.
- 잠금: `startBlock`, `blockedUntil`, `isLocked`
- 웹이 넘긴 정책표와 공부 카드: `setPolicyAndCards`
- 공부 카드 간격 복습: `nextCard`, `recordCardAnswer`, `cardDueAt` (틀림 1시간, 맞힘 1·3·7일)
- 차례대로 고르기(`nextInTurn`), 질림 반영용 마지막 표시 시각(`markShown`)

GuardPolicy.java (354줄) — 정책표 읽기

웹이 넘긴 JSON 정책표를 Java가 쓰기 좋은 모양으로 바꾼다. 목표, 단계 기준, 방식 목록, 약속 선택지, AI 표(칸별 Beta 값), 위험 시간, 공부 모드 등. 정책표가 없거나 깨지면 기본값으로 동작한다.

GuardBandit.java (약 110줄) — AI가 방식을 고르는 곳

톰슨 샘플링을 계산한다. 쉽게 말하면:

1. 방식마다 "성공 몇 번, 실패 몇 번"이 있다 (웹이 계산해서 정책표로 보냄).
2. 그 기록으로 "이 방식이 먹힐 확률은 대략 이쯤"을 **주사위처럼 한 번씩 뽑는다**. 기록이 적으면 뽑히는 값이 들쭉날쭉하다.
3. 가장 큰 값이 나온 방식을 고른다. → 잘 먹힌 방식이 자주 뽑히지만, 덜 해 본 방식도 가끔 뽑혀서 계속 배운다.
4. 방금 나온 방식은 뽑힌 값에 0.7을 곱해 잠깐 덜 고른다 (질림 반영, `freshness`).

9. 그 밖의 Java 파일

파일	하는 일
GuardPlugin.java	웹과 네이티브를 잇는 플러그인 (6부 참고)
GuardNotifier.java	잠금 중 알림창에 남은 시간 카운트다운
GuardTime.java	날짜 계산 (자정 넘긴 시청을 날짜별로 나누기 등)
MainActivity.java	앱 아이콘을 누르면 열리는 화면. Capacitor가 웹 화면을 띄운다 (12줄)

5부. 앱 화면 쪽 코드 (TypeScript) 파일별 설명

10. lib/ — 계산, 저장, AI (화면 없음)

파일	하는 일
types.ts	모든 데이터의 "모양" 정의. 설정(Settings), 시청 기록(UsageLog), 개입 기록(InterventionLog), 공부 카드(StudyCard) 등. 다른 파일이 이 모양을 지켜야 한다
logic.ts	핵심 계산 모음: 오늘 날짜, 목표 대비 비율과 단계(`levelForUsage`), 시간 체감 문구, 통계(`statsForDate`), 리포트 문장, 네이티브 기록 변환(`convertNativeBatch`), 30분 안 재진입 판정(`settleReentry`), 설정 값 검사
ai.ts	AI 계산: 밴딧 표(`banditTables`: 점수, 칸 사이 공유, 최근 가중치, 평일/주말), 위험 시간(`riskHours`), 주간 목표 제안(`goalSuggestion`)
policy.ts	정책표 만들기(`buildPolicy`), 공부 카드 넘기기(`cardsForService`), 카드 만들기 질문 문장(`CARD_PROMPT`), 붙여넣은 카드 읽기와 검사(`parseCardImport`)
storage.ts	기본 설정값(`defaultSettings`: 목표 20분 등), 저장된 값을 안전하게 읽기(틀린 줄만 버리기)
db.ts	SQLite 데이터베이스: 표 만들기, 읽기(`loadAll`), 쓰기(`applyOps`), 버전 올라갈 때 칸 추가
persist.ts	저장 방식 고르기: 폰이면 SQLite, 브라우저면 localStorage
notify.ts	알림: 매일 리포트, 위험 시간 알림과 [1시간 잠그기] 버튼
guard.ts / guard-web.ts	플러그인 `HanbakjaGuard`의 함수 목록 정의 / 브라우저용 가짜 버전
app-info.ts	앱 버전, 릴리스 주소
clips.ts	브라우저 연습 화면에서 쓰는 가짜 영상 문구 (실제 YouTube 아님)
***.test.ts**	위 계산들이 맞는지 확인하는 자동 테스트 (웹 54개)

11. components/ — 화면 조각

파일	하는 일
store.tsx	**앱 데이터의 중앙 창고.** 모든 화면이 여기서 데이터를 꺼내 쓰고, 바꿀 때도 여기를 거친다(`saveSettings`, `addCards`, `mergeNative` 등). 바뀐 내용은 persist/db로 저장
guard-bridge.tsx	**화면은 없고 1.5초마다 네이티브와 동기화**하는 부품 (3부 ⑧⑨). 위험 시간 알림 예약도 여기서
app-shell.tsx	모든 페이지 공통 틀: 위 제목, 아래 메뉴(홈, 통계, 리포트, 설정)
home-view.tsx	홈: 오늘 사용량 게이지, 시간대별 막대, 한 줄 요약, 쉰 시간
stats-view.tsx / report-view.tsx	통계, 리포트 화면
settings-view.tsx	설정 화면 (목표, 시각 등). AI 패널과 공부 카드 패널을 포함
ai-panel.tsx	AI 켜기/끄기, 엄격 모드, AI가 배운 것
cards-panel.tsx	공부 카드: 설명, 공부 모드, 직접 만들기, 붙여넣기, 확인 목록, 카드 목록
guard-panel.tsx	개입 스위치와 "정말 끌까요?" 확인 (3초 대기)
goal-suggestion.tsx	홈의 주간 목표 줄이기 제안 카드
watch-view.tsx / intervention-flow.tsx	앱 화면 안의 개입 화면. Instagram 릴스(예전 방식)와 브라우저 연습용
use-now.ts, pwa-register.tsx	현재 시각을 주기적으로 갱신 / 브라우저 설치용 도우미
ui/	버튼, 입력칸, 스위치 같은 기본 부품 (shadcn/ui)

12. app/ — 페이지 주소

Next.js는 폴더 이름이 곧 주소다. `app/settings/page.tsx` → `/settings` 주소. 각 page.tsx는 짧고, 실제 내용은 components의 view 파일을 불러 쓴다. `layout.tsx`는 모든 페이지를 감싸는 틀(글꼴, store, guard-bridge 연결)이다.

React 코드 읽는 법 (짧은 예)

```
export function CardsPanel() {                      // 화면 조각 하나 = 함수 하나
  const { data, saveSettings } = useStore();         // 중앙 창고에서 데이터 꺼내기
  const [pasted, setPasted] = useState("");          // 이 화면만의 기억 (붙여넣은 글)
  ...
  return (
    <Switch checked={data.settings.studyMode}         // 화면에 그릴 모양 (HTML 비슷)
      onCheckedChange={(on) => saveSettings({ ...data.settings, studyMode: on })} />
  );                                                 // 스위치를 누르면 설정 저장 → 화면 자동 갱신
}
```

6부. 둘을 잇는 방법

13. 플러그인 HanbakjaGuard

웹(TypeScript)에서 `HanbakjaGuard.sync(...)` 처럼 부르면, Capacitor가 Java의 `GuardPlugin.java` 안 같은 이름 함수를 실행하고 결과를 돌려준다. (이름은 팀원 원래 앱에서 이어받아 HanbakjaGuard다.)

함수	방향	하는 일
getStatus	웹 ← 네이티브	접근성 켜짐 여부, 보는 중인지, 잠금 끝나는 시각, 앱 버전
sync	웹 ↔ 네이티브	웹이 정책표와 공부 카드를 주고, 네이티브가 쌓인 기록(대기열)을 돌려줌
ack	웹 → 네이티브	"이 기록들 저장했으니 지워도 돼"
setBlocker	웹 → 네이티브	개입 스위치 켜기/끄기
startBlock / finish	웹 → 네이티브	잠금 시작 (위험 시간 알림 버튼 등) / 앱 화면 개입 결과 전달
resetRecords	웹 → 네이티브	기록 초기화
openSettings	웹 → 네이티브	폰의 접근성 설정 화면 열기

14. 왜 "대기열 + ack" 방식인가

네이티브가 기록을 바로 웹으로 보내면, 앱 화면이 꺼져 있을 때 기록을 잃는다. 그래서 네이티브는 기록을 **메모장(대기열)에 쌓아 두고**, 웹이 데이터베이스에 **저장에 성공한 것만** ack로 지운다. 저장에 실패하면 다음 sync 때 다시 받는다. → 기록이 사라지지 않는다.

15. 정책표 (웹 → 네이티브) 예시

```
{
  "version": "ai-2",                       // 어떤 계산으로 만든 표인지 (기록에 남음)
  "goalMinutes": 20,                       // 하루 목표
  "mediumAt": 0.5, "strongAt": 1,          // 목표의 50%부터 중간, 100%부터 강함
  "methods": { "light": ["pause","framing"], "medium": ["breath","alternative",...] },
  "commitOptions": [3,5,10], "extensionLadder": [10,5,3],
  "dayEndHour": 6,                         // "오늘은 끝" 풀리는 시각
  "studyMode": false,
  "ai": {
    "enabled": true, "strict": false,
    "methods": { "2:night:weekday": { "breath": [4.2, 1.8], "card": [1.5, 3.1] } },
                                           // 칸 → 방식 → [성공 쪽 값, 실패 쪽 값]
    "riskHours": [23], "riskLabel": "밤 11시",
    "habituation": { "strength": 0.3, "recoveryHours": 6 }
  }
}
```

7부. 데이터는 어디에 어떻게 저장되나

16. 두 군데에 저장한다

장소	누가	무엇을
SQLite (폰 안 데이터베이스)	웹	**진짜 기록 보관소.** 시청 기록, 개입 기록, 잠금 중 시도, 공부 카드, 설정
SharedPreferences	네이티브	잠깐 들고 있는 것: 대기열, 잠금 시각, 정책표 사본, 카드 사본, 복습 일정

17. SQLite 표 5개 (lib/db.ts)

표	한 줄 =	주요 칸
sessions	쇼츠에 머문 한 번	시작·끝 시각, 초, 날짜, 대상(youtube 등)
interventions	개입 한 번	시각, 요일, 단계, 방식, 이유, 결과(나감/봄/잠금…), 약속 분, 카드 정답 여부, 30분 안 재진입, 피드백, 정책 버전
lock_attempts	잠금 중 쇼츠를 연 한 번	시각, 남은 초
cards	공부 카드 한 장	문제, 보기(JSON), 정답 번호, 해설, 나온 횟수, 맞힌 횟수
settings	설정 묶음	목표, 시각, AI 켜기, 엄격 모드, 공부 모드 등 (JSON 한 덩어리)

개인정보: 저장하는 건 시각, 길이, 고른 버튼 같은 숫자와 선택지뿐이다. 화면 글자, 영상 제목, 메시지, 계정은 어디에도 저장하지 않는다. 외부 서버로 보내는 코드도 없다.

8부. 설정 파일, 빌드, 테스트

18. 주요 설정 파일

파일	내용
package.json	웹 라이브러리 목록, 명령어(`npm test`, `npm run apk:debug`), **앱 버전**("version": "0.9.2"). APK 버전도 여기서 가져간다
capacitor.config.ts	앱 ID(`app.focuson.shorts`), 앱 이름, 웹 빌드 폴더(`out`)
next.config.ts	Next.js를 정적 파일로 내보내는 설정
android/app/build.gradle	APK 버전 번호 계산(0.9.2 → 902), 서명 설정(keystore.properties가 있으면 릴리스 서명)
AndroidManifest.xml	앱 등록증: 접근성 서비스 등록, 알림 권한, 필요 없는 권한(생체 인증) 제거, YouTube·Instagram 앱 확인 허용
res/xml/accessibility_service_config.xml	접근성 서비스 설정: 화면 변경 이벤트 받기, 이름표 보고 받기, 작은 창 확인용 창 목록
.gitignore	git에 올리지 않을 것: node_modules, 빌드 결과, APK, 서명 키, 비밀 파일

19. 빌드: 코드에서 APK까지

명령 하나 `npm run apk:debug`가 `scripts/build-apk.mjs`를 실행해 아래를 차례로 한다.

```
1. npm run build        → Next.js가 웹 화면을 out/ 폴더에 정적 파일로 만듦
2. npx cap sync android → out/ 내용을 android/app/src/main/assets/public/ 으로 복사
                          (+ 플러그인 연결 정보 갱신)
3. gradlew assembleDebug → Gradle이 Java 코드 + 웹 파일을 묶어 APK 생성
4. release/shortsai-v0.9.2-debug.apk 로 복사, 크기와 SHA-256 출력
```

주의: 디버그 APK는 그 PC의 디버그 키로 서명된다. 다른 PC에서 빌드하면 폰에서 업데이트가 안 된다 (키 파일을 옮기거나 정식 서명 키를 만들어야 함). 그리고 새 버전을 낼 때는 package.json의 version을 꼭 올린다.

20. 테스트

종류	실행	위치와 내용
웹 테스트 54개	`npm test`	`lib/logic.test.ts`, `lib/ai.test.ts`: 날짜 계산, 단계, 통계, 기록 변환, AI 점수와 공유, 카드 가져오기 검사
Java 테스트 27개	`android/gradlew.bat testDebugUnitTest`	`android/app/src/test/...`: 쇼츠 판별, 날짜, 정책, 문구, 밴딧, 복습 일정
타입 검사	`npx tsc --noEmit`	TypeScript 자료형 실수 찾기
폰 확인	직접	`release/NOTES.md`의 버전별 확인 목록

테스트는 "이 입력이면 이 결과가 나와야 한다"를 적어 둔 것이다. 예: "목표 20분에 오늘 10분 봤으면 중간 단계여야 한다". 코드를 고친 뒤 테스트가 실패하면 어딘가 잘못 고친 것이다.

9부. 자주 하는 수정은 어디를 고치나

하고 싶은 것	고칠 곳
개입 화면 문구 바꾸기	`android/.../GuardCopy.java` (일부 버튼 글자는 `GuardFlow.java`). 바꾸기 전에 팀에서 상황별 표로 확인
앱 화면(설정, 홈) 문구 바꾸기	`components/` 의 해당 view 파일 (예: 설정 → `settings-view.tsx`, 공부 카드 → `cards-panel.tsx`)
기본 목표 시간 바꾸기 (지금 20분)	`lib/storage.ts → defaultSettings()`
단계 기준 바꾸기 (50%, 100%)	`lib/policy.ts → buildPolicy()` 의 mediumAt, strongAt (+ `lib/logic.ts → levelForUsage`)
약속 선택지 바꾸기 (3·5·10분)	`lib/policy.ts` 의 commitOptions, extensionLadder
잠금 시간 선택지 (10·20·30·60분)	`lib/logic.ts → BLOCK_MINUTES` 와 `GuardFlow.java` 의 강함 단계
대체 행동 목록 바꾸기	`GuardCopy.java → ALTERNATIVES` (웹 쪽 `lib/logic.ts → ALTERNATIVES` 도 같이)
AI 수치 조정 (공유 강도, 반감기, 점수 기준)	`lib/ai.ts` 맨 위 상수들 (`BANDIT_SHARE_K`, `BANDIT_HALF_LIFE_DAYS`, `REWARD_WATCH_CAP_MINUTES`, `HABITUATION`)
위험 시간 조이기 수치	`lib/policy.ts` 의 `risk: { commitMax, extensionMax, overdraftWaitSec }`
공부 카드 복습 간격	`GuardState.java → cardDueAt()`
카드 만들기 질문 문장	`lib/policy.ts → CARD_PROMPT`
쇼츠 판별 규칙 (YouTube가 바뀌었을 때)	`GuardScreens.java` (+ `GuardScreensTest.java` 로 확인)
새 버전 내기	`package.json` 의 version 올리기 → `npm run apk:debug` → `release/NOTES.md` 에 확인 목록 추가

고친 뒤 확인 순서:

1. `npm test` 와 `npx tsc --noEmit` (웹), `gradlew.bat testDebugUnitTest` (Java)
2. `npm run apk:debug` 로 APK 만들기
3. 폰에 업데이트로 설치해서 직접 확인
4. git 커밋

기능 설명은 "ShortsAI 기능 총정리", 기술 선택 이유는 "ShortsAI 개발 스펙", 지난 결정과 문구 표는 `docs/HANDOFF.md` 를 참고.